Task 3: Build and Train a Neural Network

Daniel Nickolov

Objectives

- Build and train a neural network
- Find a model that performs at least as well as the PC logistic regression from task 1
- Quantify the predictive accuracy

Setup

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidymodels)
```

```
## -- Attaching packages --------------------------------------- tidymodels 1.5.0 --
## v broom        1.0.13     v rsample      1.3.2
## v dials        1.4.4      v tailor       0.1.0
## v infer        1.1.0      v tune         2.1.0
## v modeldata    1.6.0      v workflows    1.3.0
## v parsnip      1.6.1      v workflowsets 1.1.1
## v recipes      1.4.0      v yardstick    1.4.0
## -- Conflicts ------------------------------------------ tidymodels_conflicts() --
## x scales::discard() masks purrr::discard()
## x dplyr::filter()   masks stats::filter()
## x recipes::fixed()  masks stringr::fixed()
## x dplyr::lag()      masks stats::lag()
## x yardstick::spec() masks readr::spec()
## x recipes::step()   masks stats::step()
```

```
library(keras3)
```

```
##
## Attaching package: 'keras3'
##
## The following object is masked from 'package:yardstick':
##
##     get_weights
##
## The following object is masked from 'package:infer':
##
##     generate
```

Load data and partition

```
load('../data/claims-clean-example.RData')

set.seed(110122)
partitions <- claims_clean %>%
  initial_split(prop = 0.8)

train_text <- training(partitions) %>%
  pull(text_clean)
train_labels <- training(partitions) %>%
  pull(bclass) %>%
  as.numeric() - 1

test_text <- testing(partitions) %>%
  pull(text_clean)
test_labels <- testing(partitions) %>%
  pull(bclass) %>%
  as.numeric() - 1
```

Preprocess Layer

```
preprocess_layer <- layer_text_vectorization(
  standardize = NULL,
  split = 'whitespace',
  ngrams = NULL,
  max_tokens = NULL,
  output_mode = 'tf_idf'
)

preprocess_layer %>% adapt(train_text)
```

NN Architecture

```
inputs <- layer_input(shape = 1, dtype = 'string')

outputs <- inputs %>%
```

```
  preprocess_layer() %>%
  layer_dropout(0.2) %>%
  layer_dense(units = 25, activation = 'relu') %>%
  layer_dropout(0.2) %>%
  layer_dense(1, activation = 'sigmoid')

model <- keras_model(inputs, outputs)

# Use ASCII output so the model table can compile with pdflatex.
summary(model, print_fn = function(x, ...) {
  cat(iconv(x, from = "UTF-8", to = "ASCII", sub = "-"), "\n")
})
```

```
## Model: "functional"
## ---------------------------------------------------------------------------------------------------
## --- Layer (type)                        --- Output Shape              ---       Param # ---
## ---------------------------------------------------------------------------------------------------
## --- input_layer (InputLayer)            --- (None, 1)                 ---             0 ---
## ---------------------------------------------------------------------------------------------------
## --- text_vectorization                  --- (None, 1, 34032)          ---             0 ---
## --- (TextVectorization)                 ---                           ---               ---
## ---------------------------------------------------------------------------------------------------
## --- dropout (Dropout)                   --- (None, 1, 34032)          ---             0 ---
## ---------------------------------------------------------------------------------------------------
## --- dense (Dense)                       --- (None, 1, 25)             ---       850,825 ---
## ---------------------------------------------------------------------------------------------------
## --- dropout_1 (Dropout)                 --- (None, 1, 25)             ---             0 ---
## ---------------------------------------------------------------------------------------------------
## --- dense_1 (Dense)                     --- (None, 1, 1)              ---            26 ---
## ---------------------------------------------------------------------------------------------------
##  Total params: 850,851 (3.25 MB)
##  Trainable params: 850,851 (3.25 MB)
##  Non-trainable params: 0 (0.00 B)
##
```

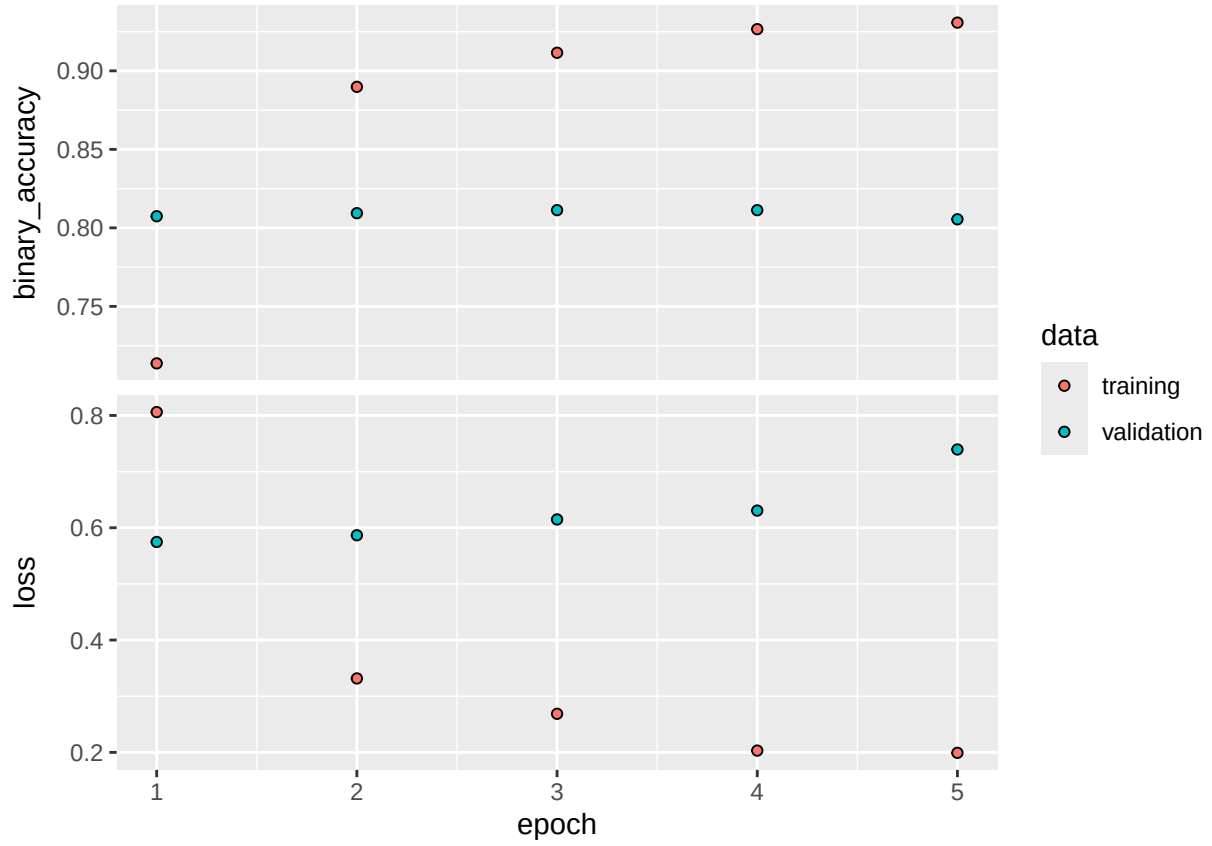
Training the model

```
# configure for training
model %>% compile(
  loss = 'binary_crossentropy',
  optimizer = 'adam',
  metrics = 'binary_accuracy'
)

# train
history <- model %>%
  fit(train_text,
      train_labels,
      validation_split = 0.3,
      epochs = 5)
```

```
## Epoch 1/5
## 38/38 - 0s - 11ms/step - binary_accuracy: 0.7137 - loss: 0.8060 - val_binary_accuracy: 0.8074 - val_l
## Epoch 2/5
## 38/38 - 0s - 4ms/step - binary_accuracy: 0.8898 - loss: 0.3316 - val_binary_accuracy: 0.8093 - val_lo
## Epoch 3/5
## 38/38 - 0s - 4ms/step - binary_accuracy: 0.9115 - loss: 0.2687 - val_binary_accuracy: 0.8113 - val_lo
## Epoch 4/5
## 38/38 - 0s - 4ms/step - binary_accuracy: 0.9265 - loss: 0.2031 - val_binary_accuracy: 0.8113 - val_lo
## Epoch 5/5
## 38/38 - 0s - 4ms/step - binary_accuracy: 0.9307 - loss: 0.1992 - val_binary_accuracy: 0.8054 - val_lo
```

```
plot(history)
```



Checking Test Set Accuracy

```
evaluate(model, test_text, test_labels)
```

```
## 14/14 - 0s - 2ms/step - binary_accuracy: 0.8084 - loss: 0.9857

## $binary_accuracy
## [1] 0.8084112
##
## $loss
## [1] 0.9856806
```

```r
# predictions and confusion matrix
test_probs <- predict(model, test_text) %>% as.numeric()
```

```
## 14/14 - 0s - 4ms/step
```

```r
test_results <- tibble(
  truth = factor(test_labels, levels = c(0, 1)),
  pred = factor(as.numeric(test_probs > 0.5), levels = c(0, 1))
)

accuracy(test_results, truth = truth, estimate = pred)
```

```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.808
```

```r
conf_mat(test_results, truth = truth, estimate = pred)
```

```
##           Truth
## Prediction   0   1
##          0 161  47
##          1  35 185
```

Comparing this to our accuracy in preliminary task 1 of about 77.38%, we notice a close to 5% improvement in prediction accuracy running the neural network over the PC logistic regression, getting about 0.82% accuracy here.

Save model

```r
save_model(model, '../results/task3-model.keras')
```